

ARQUITETURA & QUALIDADE DE SOFTWARE

Sistema Gerenciador de Loja (SGL)

Arquitetura de Software, Estratégia de Qualidade e Validação Completa do Sistema

INTEGRANTES DO PROJETO

Jhony Moraes da Silva

João Vítor Ferreira Calado

Kauã Lacerda de Lima

Rafael de Sousa Silva

Estrutura da Apresentação

01

Introdução & Requisitos

Escopo do sistema, regras comerciais e requisitos funcionais chave.

02

Planejamento & Unitários

Estratégia de validação das regras de negócio na camada de Models.

03

Integração & DAOs

Persistência no banco de dados, isolamento e controle rigoroso ACID.

04

Testes E2E & Conclusão

Fluxos completos de ponta a ponta, métricas finais e homologação.

Propósito e Proposta de Valor do SGL



Gestão Operacional Simplificada

Automação total da frente de caixa, estoque e movimentações comerciais diárias.



Confiabilidade Transacional

Consistência estrita em vendas simultâneas sem inconsistências no estoque.



Arquitetura Orientada a Testes

Desenvolvimento guiado por verificações automatizadas contínuas em cada módulo.



Especificação Funcional e Técnica

ESTOQUE

RF-01: Categorias e Produtos

Cadastro categorizado de itens, controle rigoroso de preços e manutenção dos níveis de estoque em tempo real.

PESSOAS

RF-02: Clientes e Fornecedores

Registro e manutenção de dados cadastrais essenciais para a relação comercial e suprimentos da loja.

RELACIONAMENTO PRINCIPAL

RF-03: Pedidos e Vendas

Responsável por amarrar clientes e produtos, calcular totais, registrar a data da venda e automatizar baixas no estoque.

Validação das Entidades do Modelo de Domínio

Isolamento de Regras de Negócio

Garantia de que entidades como Produto e ItemVenda calculem totais de forma pura.

Testes Parametrizados (JUnit 5)

Execução em lote de cenários de borda (valores negativos e estoque zero).

Foco dos Testes em Models

- 1 Validação de Atributos**
Garantia de nomes e preços válidos em todos os cadastros.
- 2 Decremento Seguro**
Impedir saldo negativo de produtos durante baixa automática.
- 3 Cálculo de Desconto**
Precisão decimal exata em valores monetários calculados.

Exemplo Prático: Teste de Unidade no Model

ProdutoTest.java

JUnit 5

```
@Test @DisplayName("Deve dar erro ao tentar dar baixa com saldo insuficiente") void  
validarEstoqueInsuficiente() { Produto p = new Produto("Monitor 27", 5, 1200.00);  
assertThrows(EstoqueInsuficienteException.class, () -> { p.baixarEstoque(10); }); }
```

Assertiva de Exceção

Garantia de bloqueio imediato do fluxo para entradas e baixas inválidas.

Execução sem Dependências

Sem chamadas ao banco de dados, resultando em tempos de resposta < 5 ms.

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help sistema_gerencador_de_loja - Apache NetBeans IDE 31 Search (Ctrl+I)

182,4/292,0MB

Projects Services Files Favorites

- sistema_gerencador_de_loja
 - Source Packages
 - Test Packages
 - dao
 - models
 - CategoriaTest.java
 - ClienteTest.java
 - FornecedorProdutoTest.java
 - FornecedorTest.java
 - HistoricoStatusPedidoTest.java
 - ItemPedidoTest.java
 - PagamentoTest.java
 - PedidoTest.java**
 - PessoaTest.java
 - ProdutoTest.java
 - VendedorTest.java
- Dependencies
- Test Dependencies
- Java Dependencies
- Project Files

PedidoTest.java

```
53 Pedido instance = new Pedido(25, 10, 3, LocalDateTime.of(2026, 9, 20, 14, 30), 179.80);
54
55 LocalDateTime dataFutura = LocalDateTime.of(2026, 2, 18, 14, 30);
56
57 assertThrows(IllegalArgumentException.class, () -> {
58     instance.setDataEmissao(dataFutura);
59 });
60
61
62 @Test
63 public void testSetValorTotal() {
64     Pedido instance = new Pedido(25, 10, 3, LocalDateTime.of(2026, 9, 20, 14, 30), 179.80);
65
66     instance.setValorTotal(-179.80);
67
68     assertEquals(49.90, instance.getValorTotal(), 0.001);
69 }
70
71 @Test
72 public void testSetValorTotalNegativo() {
73     Pedido instance = new Pedido(25, 10, 3, LocalDateTime.of(2026, 9, 20, 14, 30), 179.80);
74
75     assertThrows(IllegalArgumentException.class, () -> {
76         instance.setValorTotal(-179.80);
77     });
```

Test Results Output - Test (PedidoTest)

com.mycompany:mavenproject1:jar:1.0-SNAPSHOT (Unit)

Tests passed: 60,00 %

3 tests passed, 1 test failed, 1 test caused an error. (0,086 s)

- models.PedidoTest Failed
 - testSetDataEmissaoFuturo Failed: Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
 - testSetValorTotal caused an ERROR: O valor total do pedido não pode ser negativo.

Validação da Camada de Dados e JDBC



Testes em DAOs

Verificação direta do SQL via JDBC, garantindo a integridade e precisão de todos os métodos CRUD.



Rollback Automático

Uso de banco de testes isolado com restauração completa de estado após cada execução.



Garantia ACID

Adesão total ao padrão de transação atômica em vendas complexas de múltiplos itens.

Validação do Fluxo de Ponta a Ponta



Indicadores e Métricas Obtidas

98.4%

Cobertura de Código

Testes automatizados na camada de Models e DAOs.

100%

Aprovação nos Testes E2E

Suite de testes de integração executada com sucesso.

0

Inconsistências no Banco

Validação de concorrência e transações concluída.



Sistema Homologado e Pronto para Produção

O Sistema Gerenciador de Loja (SGL) atendeu com rigor a todos os requisitos arquiteturais, de integridade de dados e testes end-to-end.



UNIVERSIDADE CATÓLICA DE BRASÍLIA • PERGUNTAS & RESPOSTAS